ROS 2 Actions

Complete Tutorial

Robot Grid Pathfinder

with Live Step-by-Step Movement Feedback

What you will build:

A ROS 2 action server that receives a start cell and end cell on a 10x10 grid, computes the shortest path using BFS, then **executes each movement step one at a time**—publishing live feedback (UP, DOWN, LEFT, RIGHT, position, steps remaining) after every move. The client displays each step as it happens and draws the final grid map.

The action can be **cancelled mid-navigation** by the client.

Builds on top of: `pathfinder_interfaces` and `pathfinder_nodes`

The same packages used in the Service & Client tutorial. We add a `.action` file and two new node files.

Environment

Item	Version
ROS 2	Humble Hawksbill
OS	Ubuntu 22.04 LTS
Python	3.10+
Build Tool	colcon
Algorithm	Breadth-First Search (BFS)
New concept	ROS 2 Actions (Goal / Feedback / Result)

Contents

1 What Are ROS 2 Actions? 3

2 Actions vs Services — A Clear Comparison 3
2.1 The Mental Model . . . 4
2.2 Pathfinder: Service vs Action Side by Side . . . 5

3 What This Tutorial Builds 5
3.1 The .action File Structure . . . 6

4 Package Structure 6

5 Create the Action Definition 7
5.1 Create the action folder and file . . . 7
5.2 Write the .action file . . . 7
5.3 Update CMakeLists.txt . . . 8
5.4 Build and verify the interface . . . 8

6 Write the Action Server 9
6.1 Key Parts of the Server Explained . . . 13

7 Write the Action Client 13
7.1 Key Parts of the Client Explained . . . 17

8 Update setup.py 17

9 Build and Run 18
9.1 Build both packages . . . 18
9.2 Terminal 1 — Start the action server . . . 18
9.3 Terminal 2 — Run the action client . . . 18

10 Expected Live Output 18

11 Test Cases 19
11.1 Test 1 — Standard navigation (original example) . . . 19
11.2 Test 2 — Corner to corner . . . 19
11.3 Test 3 — Short path, same row . . . 19
11.4 Test 4 — Navigate around central obstacle . . . 19
11.5 Test 5 — Start on an obstacle (aborted goal) . . . 20
11.6 Test 6 — End on an obstacle (aborted goal) . . . 20
11.7 Test 7 — Same start and end (zero steps) . . . 20
11.8 Test 8 — Out of bounds (aborted goal) . . . 20
11.9 Test 9 — CLI test without client node . . . 20
11.10 Test 10 — Vertical navigation . . . 21

12 CLI Tools for Actions 21

13 Complete Communication Flow 21

14 Topics vs Services vs Actions — Full Summary **23**

15 Common Mistakes & Fixes **23**

16 Quick Reference **23**

16.1 Commands Cheat Sheet 23

16.2 Test Cases Summary 24

1 What Are ROS 2 Actions?

In ROS 2, nodes exchange information using three mechanisms: **topics**, **services**, and **actions**. Each fills a different role. Topics are one-way streams of continuous data — a sensor publishing readings sixty times per second, a camera streaming frames, a wheel encoder broadcasting odometry. Services are two-way, instant calls — send a request, receive a reply in milliseconds, done. Actions are the third mechanism, designed specifically for operations that take a **meaningful amount of time** and need to communicate progress along the way.

An action is built around three pieces of information. The **goal** is what the client sends at the start to tell the server what to do. The **feedback** is what the server sends repeatedly while it is working, providing live updates on progress. The **result** is what the server sends exactly once when the operation is fully complete. Unlike a service, which blocks until it is done and returns only a single response, an action runs asynchronously in the background while the client receives a continuous stream of updates. And unlike a topic, an action has a defined start, a defined end, a structured goal, and can be cancelled by the client at any point.

This makes actions the natural choice for anything that resembles a task: navigate to this waypoint, pick up this object, scan this area, charge to 80%, execute this joint trajectory. All of these take time, all of them benefit from progress feedback, and all of them might need to be stopped before they finish.

2 Actions vs Services — A Clear Comparison

The most important distinction to understand is **when to use an action versus a service**. The table below summarises the key differences:

	Service	Action
Duration	Completes in milliseconds	Takes seconds, minutes, or longer
Response timing	Single response at the end	Continuous feedback while running, plus a final result
Cancellable	No — client must wait	Yes — client can cancel at any time
Blocking	Client blocks until reply arrives	Client sends goal and continues; feedback arrives asynchronously
Communication structure	Request / Response (2 parts)	Goal / Feedback / Result (3 parts)
File type	`.srv` file	`.action` file
ROS 2 library	`create_service` / `create_client`	`ActionServer` / `ActionClient`
Typical use	Instant queries: convert temperature, check battery level	Long tasks: navigate to point, execute arm trajectory
This tutorial	Return the full path list at once	Execute each step with live direction feedback

2.1 The Mental Model

```
Topic
  Publisher continuously sends data.
  Subscribers listen. No reply. No start. No end.
  Like a radio broadcast.

Service
  Client sends a request.
  Server processes it instantly.
  Server sends one response.
  Client was blocked the whole time.
  Like sending a text and waiting for an immediate reply.

Action
  Client sends a goal ("navigate from A to B").
  Server accepts the goal and starts working.
  Server sends feedback every step ("moved RIGHT, now at (3,2)").
  Server sends feedback every step ("moved DOWN, now at (3,3)").
  ...
  Server sends final result ("reached (8,6) in 14 steps").
  Client can cancel at any point.
  Like tracking a delivery in real time.
```

2.2 Pathfinder: Service vs Action Side by Side

To make this concrete, here is how the same pathfinding problem looks as a service versus as an action:

Aspect	As a Service	As an Action
Client sends	start, end coordinates	start, end coordinates (goal)
Server computes	BFS path immediately	BFS path, then executes it
Client receives	Full path list at once	One step at a time, live
Feedback	None	Direction, position, steps left
Robot experience	Teleports to the end	Moves step by step visibly
Cancel possible	No	Yes, at any step
Real-world match	Path planning only	Path planning + execution

Key Insight

Use a **service** when you need a computed answer. Use an **action** when you need to execute a task and watch it happen. In a real robot system you would often use *both* — a service to plan the path, then an action to execute it.

3 What This Tutorial Builds

This tutorial extends the grid pathfinder built in the previous Service tutorial. The grid, the obstacle map, and the BFS algorithm remain exactly the same. What changes is **what happens after the path is found**.

In the service version, the server found the path and immediately returned the full list of coordinates to the client. The client printed the steps and drew the grid. There was no sense of movement — just a list of cells.

In this action version, the server finds the path and then **executes it step by step**, pausing between each move to simulate real robot movement. After each step it publishes a feedback message to the client containing:

- The current position `(x, y)`
- The step number (e.g. Step 3 of 14)
- How many steps remain
- The direction taken to reach this cell: **UP**, **DOWN**, **LEFT**, or **RIGHT**
- A status string describing the current state

The client prints each feedback message as it arrives, one line every 0.5 seconds, so the operator can watch the robot move in real time. When the robot reaches the destination the server sends a final result containing success status, total steps taken, and the full path. The client then draws the ASCII grid showing the completed path.

If the client sends a cancel request at any point during navigation, the server detects it, stops immediately, and returns a result explaining where it stopped.

3.1 The .action File Structure

Every `.action` file has exactly three sections separated by `---`:

NavigatePath.action structure

```
[GOAL SECTION]
int32 start_x
int32 start_y
int32 end_x
int32 end_y
---
[RESULT SECTION]
bool    success
string  message
int32   total_steps
int32[] path_x
int32[] path_y
---
[FEEDBACK SECTION]
int32  current_x
int32  current_y
int32  step_number
int32  steps_remaining
string direction
string status
```

Section	Sent by	When
Goal	Client	Once, at the start
Feedback	Server	Repeatedly, after each step
Result	Server	Once, when fully complete or cancelled

4 Package Structure

This tutorial adds files to the existing packages from the Service tutorial. No new packages are needed.

```
ros2_ws/
 src/
     pathfinder_interfaces/         <- ament_cmake
        srv/
           FindPath.srv           <- from service tutorial
        action/
           NavigatePath.action    <- NEW
        CMakeLists.txt             <- updated
        package.xml
```

```
pathfinder_nodes/                <- ament_python
    pathfinder_nodes/
        __init__.py
        pathfinder_server.py    <- from service tutorial
        pathfinder_client.py    <- from service tutorial
        navigate_server.py      <- NEW
        navigate_client.py      <- NEW
    package.xml
    setup.cfg
    setup.py                     <- updated
```

5 Create the Action Definition

5.1 Create the action folder and file

```
cd ~/ros2_ws/src/pathfinder_interfaces
mkdir action
touch action/NavigatePath.action
```

5.2 Write the .action file

Open `action/NavigatePath.action` and write exactly this — **no comments, no blank lines**:

NavigatePath.action

```
int32 start_x
int32 start_y
int32 end_x
int32 end_y
---
bool success
string message
int32 total_steps
int32[] path_x
int32[] path_y
---
int32 current_x
int32 current_y
int32 step_number
int32 steps_remaining
string direction
string status
```

No Comments in .action Files

Just like `.srv` files, avoid comment lines (`#`) in `.action` files in ROS 2 Humble. The `rosidl` parser can misinterpret them as invalid field names. Keep the file clean — field definitions only.

5.3 Update CMakeLists.txt

Listing 1: pathfinder_interfaces/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(pathfinder_interfaces)

find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "srv/FindPath.srv"
  "action/NavigatePath.action"
)

ament_package()
```

5.4 Build and verify the interface

```
cd ~/ros2_ws
colcon build --packages-select pathfinder_interfaces
source install/setup.bash

# Verify all three sections are present
ros2 interface show pathfinder_interfaces/action/NavigatePath
```

Expected Output — All 15 fields across 3 sections

```
int32 start_x
int32 start_y
int32 end_x
int32 end_y
---
bool success
string message
int32 total_steps
int32[] path_x
int32[] path_y
---
int32 current_x
int32 current_y
int32 step_number
int32 steps_remaining
```

```
string direction
string status
```

6 Write the Action Server

Listing 2: navigate_server.py

```python
import time
import rclpy
from rclpy.node import Node
from rclpy.action import ActionServer
from collections import deque
from pathfinder_interfaces.action import NavigatePath


class NavigateServer(Node):

    COLS = 10
    ROWS = 10
    STEP_DELAY = 0.5     # seconds between each movement step

    OBSTACLES = {
        (2,1),(3,1),(4,1),
        (4,2),(8,2),
        (4,3),(8,3),
        (2,4),(4,4),
        (2,5),(4,5),(5,5),(6,5),(7,5),
        (4,6),(5,6),(6,6),(7,6),
    }

    def __init__(self):
        super().__init__('navigate_server')

        self.action_server = ActionServer(
            self,
            NavigatePath,            # action type
            'navigate_path',         # action name
            self.execute_callback    # called when a goal arrives
        )

        self.get_logger().info('Navigate Action Server is ready.')
        self.get_logger().info(
            f'Grid: {self.COLS}x{self.ROWS} | '
            f'Step delay: {self.STEP_DELAY}s'
        )

    def is_valid(self, x, y):
        in_bounds = 0 <= x < self.COLS and 0 <= y < self.ROWS
        free      = (x, y) not in self.OBSTACLES
```

```
        return in_bounds and free

    def bfs(self, start, end):
        queue   = deque()
        visited = set()
        queue.append([start])
        visited.add(start)

        while queue:
            path    = queue.popleft()
            current = path[-1]
            if current == end:
                return path
            x, y = current
            for dx, dy in [(0,-1),(0,1),(-1,0),(1,0)]:
                nx, ny    = x + dx, y + dy
                neighbour = (nx, ny)
                if self.is_valid(nx, ny) and neighbour not in visited:
                    visited.add(neighbour)
                    queue.append(path + [neighbour])
        return None

    def get_direction(self, current, next_cell):
        dx = next_cell[0] - current[0]
        dy = next_cell[1] - current[1]
        if dx ==  1 and dy ==  0: return 'RIGHT'
        if dx == -1 and dy ==  0: return 'LEFT'
        if dx ==  0 and dy == -1: return 'UP'
        if dx ==  0 and dy ==  1: return 'DOWN'
        return 'UNKNOWN'

    def execute_callback(self, goal_handle):
        goal  = goal_handle.request
        start = (goal.start_x, goal.start_y)
        end   = (goal.end_x,   goal.end_y)

        self.get_logger().info(f'Goal received: {start} --> {end}')

        result = NavigatePath.Result()

        # Validate start
        if not self.is_valid(*start):
            self.get_logger().error(f'Start {start} is invalid.')
            goal_handle.abort()
            result.success     = False
            result.message     = f'Start {start} is invalid.'
            result.total_steps = 0
            return result

```

```
        # Validate end
        if not self.is_valid(*end):
            self.get_logger().error(f'End {end} is invalid.')
            goal_handle.abort()
            result.success      = False
            result.message      = f'End {end} is invalid.'
            result.total_steps = 0
            return result

        # Same cell
        if start == end:
            goal_handle.succeed()
            result.success      = True
            result.message      = 'Already at destination.'
            result.total_steps = 0
            result.path_x       = [goal.start_x]
            result.path_y       = [goal.start_y]
            return result

        # Run BFS
        path = self.bfs(start, end)

        if path is None:
            self.get_logger().error('No path found.')
            goal_handle.abort()
            result.success      = False
            result.message      = f'No path from {start} to {end}.'
            result.total_steps = 0
            return result

        total_steps = len(path) - 1
        self.get_logger().info(
            f'Path found: {total_steps} steps. Executing...'
        )

        # Execute each step and publish feedback
        feedback = NavigatePath.Feedback()

        for i in range(len(path)):

            # Check for cancellation request from client
            if goal_handle.is_cancel_requested:
                goal_handle.canceled()
                self.get_logger().warn(f'Goal cancelled at step {i}.')
                result.success      = False
                result.message      = f'Cancelled at step {i} of {total_steps}.'
                result.total_steps = total_steps
                result.path_x       = [p[0] for p in path]
```

```
                result.path_y      = [p[1] for p in path]
                return result

            current   = path[i]
            direction = 'START' if i == 0 else self.get_direction
                (
                path[i-1], path[i]
            )

            # Fill feedback fields
            feedback.current_x       = current[0]
            feedback.current_y       = current[1]
            feedback.step_number     = i
            feedback.steps_remaining = total_steps - i
            feedback.direction       = direction
            feedback.status = (
                'AT START'       if i == 0           else
                'AT DESTINATION' if i == total_steps else
                f'Moving {direction}'
            )

            # Publish feedback to client
            goal_handle.publish_feedback(feedback)

            self.get_logger().info(
                f'Step {i:2d}/{total_steps} | '
                f'Pos: ({current[0]},{current[1]}) | '
                f'Dir: {direction:7s} | '
                f'Left: {total_steps - i}'
            )

            time.sleep(self.STEP_DELAY)

        # Mark goal as succeeded and return final result
        goal_handle.succeed()

        result.success     = True
        result.message     = (
            f'Reached {end} in {total_steps} steps.'
        )
        result.total_steps = total_steps
        result.path_x      = [p[0] for p in path]
        result.path_y      = [p[1] for p in path]

        self.get_logger().info(
            f'Navigation complete! {total_steps} steps.'
        )
        return result


def main(args=None):
```

```
    rclpy.init(args=args)
    node = NavigateServer()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

6.1 Key Parts of the Server Explained

Code	Purpose
`ActionServer(self, Type, name, cb)`	Registers the action server with ROS 2
`execute_callback(goal_handle)`	Called once when client sends a goal
`goal_handle.request`	The goal object sent by the client
`goal_handle.abort()`	Tell client the goal failed
`goal_handle.succeed()`	Tell client the goal completed
`goal_handle.canceled()`	Tell client the goal was cancelled
`goal_handle.is_cancel_requested`	True if client asked to cancel
`goal_handle.publish_feedback(fb)`	Send one feedback message to client
`NavigatePath.Feedback()`	Empty feedback object to fill in
`NavigatePath.Result()`	Empty result object to fill in
`time.sleep(STEP_DELAY)`	Simulates robot movement time
`return result`	Always return result at the end

7 Write the Action Client

Listing 3: navigate_client.py

```
import sys
import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from pathfinder_interfaces.action import NavigatePath


class NavigateClient(Node):

    OBSTACLES = {
        (2,1),(3,1),(4,1),
        (4,2),(8,2),
        (4,3),(8,3),
        (2,4),(4,4),
        (2,5),(4,5),(5,5),(6,5),(7,5),
        (4,6),(5,6),(6,6),(7,6),
    }

```

```python
    def __init__(self):
        super().__init__('navigate_client')

        self.action_client = ActionClient(
            self,
            NavigatePath,        # action type -- must match server
            'navigate_path'      # action name -- must match server
        )

        self.get_logger().info('Waiting for navigate action server...')
        self.action_client.wait_for_server()
        self.get_logger().info('Action server connected!')

    def send_goal(self, start_x, start_y, end_x, end_y):
        goal          = NavigatePath.Goal()
        goal.start_x = start_x
        goal.start_y = start_y
        goal.end_x   = end_x
        goal.end_y   = end_y

        self.get_logger().info(
            f'Sending goal: ({start_x},{start_y}) --> ({end_x},{end_y})'
        )

        # send_goal_async does NOT block -- returns a future
        send_goal_future = self.action_client.send_goal_async(
            goal,
            feedback_callback=self.feedback_callback
        )

        # Wait until server accepts or rejects the goal
        rclpy.spin_until_future_complete(self, send_goal_future)
        goal_handle = send_goal_future.result()

        if not goal_handle.accepted:
            self.get_logger().error('Goal was rejected by the server.')
            return None

        self.get_logger().info('Goal accepted! Navigation in progress...')
        print()
        print('==========================================')
        print(f'  Navigating ({start_x},{start_y}) --> ({end_x},{end_y})')
        print('==========================================')

```

```
        # get_result_async waits for the final result
        result_future = goal_handle.get_result_async()
        rclpy.spin_until_future_complete(self, result_future)

        return result_future.result().result

    def feedback_callback(self, feedback_msg):
        fb = feedback_msg.feedback

        arrows = {
            'UP':      'UP   ',
            'DOWN':    'DOWN ',
            'LEFT':    'LEFT ',
            'RIGHT':   'RIGHT',
            'START':   'START',
            'UNKNOWN': '?????',
        }
        arrow = arrows.get(fb.direction, '?????')

        print(
            f'  Step {fb.step_number:2d} | '
            f'{arrow} | '
            f'Pos: ({fb.current_x},{fb.current_y}) | '
            f'Remaining: {fb.steps_remaining:2d} | '
            f'{fb.status}'
        )

    def draw_grid(self, path_x, path_y, start, end, cols=10, rows
       =10):
        path_set = set(zip(path_x, path_y))

        print()
        print('      ' + '  '.join([str(x) for x in range(cols)]))
        print('   +' + ('--+' * cols))

        for y in range(rows):
            row = f' {y} |'
            for x in range(cols):
                cell = (x, y)
                if   cell == start:              row += 'S |'
                elif cell == end:                row += 'E |'
                elif cell in self.OBSTACLES:     row += '##|'
                elif cell in path_set:           row += '. |'
                else:                            row += '  |'
            print(row)
            print('   +' + ('--+' * cols))

        print()
        print('  S=Start  E=End  .=Path  ##=Obstacle')
        print()

```

```python

def main(args=None):
    rclpy.init(args=args)
    node = NavigateClient()

    # Usage: ros2 run pathfinder_nodes navigate_client 1 1 8 6
    start_x = int(sys.argv[1])
    start_y = int(sys.argv[2])
    end_x   = int(sys.argv[3])
    end_y   = int(sys.argv[4])

    result = node.send_goal(start_x, start_y, end_x, end_y)

    if result is None:
        return

    print()
    print('========================================')
    print('          NAVIGATION COMPLETE           ')
    print('========================================')

    if result.success:
        print(f'  Status  : SUCCESS')
        print(f'  Steps   : {result.total_steps}')
        print(f'  Message : {result.message}')
        print('========================================')
        node.draw_grid(
            path_x = result.path_x,
            path_y = result.path_y,
            start  = (start_x, start_y),
            end    = (end_x, end_y)
        )
    else:
        print(f'  Status  : FAILED or CANCELLED')
        print(f'  Message : {result.message}')
        print('========================================')

    node.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

7.1 Key Parts of the Client Explained

Code	Purpose
`ActionClient(self, Type, name)`	Creates the action client
`action_client.wait_for_server()`	Blocks until server is found
`NavigatePath.Goal()`	Empty goal object to fill in
`send_goal_async(goal, feedback_cb)`	Sends goal; registers feedback callback
`goal_handle.accepted`	True if server accepted the goal
`goal_handle.get_result_async()`	Returns future for the final result
`spin_until_future_complete(...)`	Waits for future to have a value
`result_future.result().result`	Extracts the actual Result object
`feedback_callback(feedback_msg)`	Called automatically on each feedback
`feedback_msg.feedback`	The Feedback object from the server

8 Update setup.py

Listing 4: pathfinder_nodes/setup.py

```python
from setuptools import find_packages, setup

package_name = 'pathfinder_nodes'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='your_name',
    maintainer_email='you@email.com',
    description='Pathfinder service server, client, and action nodes',
    license='Apache-2.0',
    extras_require={'test': ['pytest']},
    entry_points={
        'console_scripts': [
            'pathfinder_server = pathfinder_nodes.pathfinder_server:main',
            'pathfinder_client = pathfinder_nodes.pathfinder_client:main',
            'navigate_server   = pathfinder_nodes.navigate_server:main',
            'navigate_client   = pathfinder_nodes.navigate_client:main',
```

```
        ],
    },
)
```

9 Build and Run

9.1 Build both packages

```
cd ~/ros2_ws

# Step 1 -- rebuild interfaces (new .action file added)
colcon build --packages-select pathfinder_interfaces
source install/setup.bash

# Step 2 -- rebuild nodes (new server and client added)
colcon build --packages-select pathfinder_nodes
source install/setup.bash
```

Expected Build Output

```
Starting >>> pathfinder_interfaces
Finished <<< pathfinder_interfaces [6.8s]
Summary: 1 package finished

Starting >>> pathfinder_nodes
Finished <<< pathfinder_nodes [1.9s]
Summary: 1 package finished
```

9.2 Terminal 1 — Start the action server

```
ros2 run pathfinder_nodes navigate_server
```

Server Startup Output

```
[INFO] [navigate_server]: Navigate Action Server is ready.
[INFO] [navigate_server]: Grid: 10x10 | Step delay: 0.5s
```

9.3 Terminal 2 — Run the action client

```
# Usage: navigate_client <start_x> <start_y> <end_x> <end_y>
ros2 run pathfinder_nodes navigate_client 1 1 8 6
```

10 Expected Live Output

The client prints one line every 0.5 seconds as the robot moves:

11 Test Cases

Run the server in Terminal 1, then run each test in Terminal 2.

11.1 Test 1 — Standard navigation (original example)

```
ros2 run pathfinder_nodes navigate_client 1 1 8 6
```

What to expect

Robot navigates from (1,1) to (8,6), printing each step live. You see UP, DOWN, LEFT, RIGHT directions printed one at a time, 0.5s apart. Final grid is drawn on completion.

11.2 Test 2 — Corner to corner

```
ros2 run pathfinder_nodes navigate_client 0 0 9 9
```

What to expect

Longest possible navigation on the grid. Many steps, all printed live. Tests that the action handles a long-running task correctly.

11.3 Test 3 — Short path, same row

```
ros2 run pathfinder_nodes navigate_client 0 0 5 0
```

Expected

```
Step  0 | START | Pos: (0,0) | Remaining:  5 | AT START
Step  1 | RIGHT | Pos: (1,0) | Remaining:  4 | Moving RIGHT
Step  2 | RIGHT | Pos: (2,0) | Remaining:  3 | Moving RIGHT
Step  3 | RIGHT | Pos: (3,0) | Remaining:  2 | Moving RIGHT
Step  4 | RIGHT | Pos: (4,0) | Remaining:  1 | Moving RIGHT
Step  5 | RIGHT | Pos: (5,0) | Remaining:  0 | AT DESTINATION
Steps: 5
```

11.4 Test 4 — Navigate around central obstacle

```
ros2 run pathfinder_nodes navigate_client 0 3 9 3
```

What to expect

Row 3 has obstacles at (4,3) and (8,3). The robot must navigate around them. You can see the direction changes in the live feedback.

11.5 Test 5 — Start on an obstacle (aborted goal)

```
ros2 run pathfinder_nodes navigate_client 2 1 8 6
```

Expected

```
Status  : FAILED or CANCELLED
Message : Start (2, 1) is invalid.
```

Note

The server calls `goal_handle.abort()` and returns immediately. No feedback is published because navigation never started.

11.6 Test 6 — End on an obstacle (aborted goal)

```
ros2 run pathfinder_nodes navigate_client 0 0 4 5
```

Expected

```
Status  : FAILED or CANCELLED
Message : End (4, 5) is invalid.
```

11.7 Test 7 — Same start and end (zero steps)

```
ros2 run pathfinder_nodes navigate_client 3 3 3 3
```

Expected

```
Status  : SUCCESS
Steps   : 0
Message : Already at destination.
```

11.8 Test 8 — Out of bounds (aborted goal)

```
ros2 run pathfinder_nodes navigate_client 0 0 15 15
```

Expected

```
Status  : FAILED or CANCELLED
Message : End (15, 15) is invalid.
```

11.9 Test 9 — CLI test without client node

```
# Send a goal and watch live feedback directly from terminal
ros2 action send_goal --feedback /navigate_path \
  pathfinder_interfaces/action/NavigatePath \
  "{start_x: 1, start_y: 1, end_x: 8, end_y: 6}"
```

Note

The `-feedback` flag makes the CLI print each feedback message as it arrives. Without it you only see the final result.

11.10 Test 10 — Vertical navigation

```
ros2 run pathfinder_nodes navigate_client 0 0 0 9
```

Expected

All steps should be `DOWN` — straight vertical path along column 0 which has no obstacles. 9 steps total.

12 CLI Tools for Actions

```
# List all active action servers
ros2 action list

# Show action type definition (all 3 sections)
ros2 action info /navigate_path

# Send a goal from terminal (no feedback shown)
ros2 action send_goal /navigate_path \
  pathfinder_interfaces/action/NavigatePath \
  "{start_x: 0, start_y: 0, end_x: 9, end_y: 9}"

# Send a goal AND watch live feedback in terminal
ros2 action send_goal --feedback /navigate_path \
  pathfinder_interfaces/action/NavigatePath \
  "{start_x: 1, start_y: 1, end_x: 8, end_y: 6}"

# Show interface definition
ros2 interface show pathfinder_interfaces/action/NavigatePath
```

13 Complete Communication Flow

```
ros2 run pathfinder_nodes navigate_client 1 1 8 6
          |
          v
NavigateClient.__init__()
  ActionClient(NavigatePath, 'navigate_path')
  wait_for_server()  -- server found
          |
          v
send_goal(1, 1, 8, 6)
```

```
goal.start_x=1  goal.start_y=1
goal.end_x=8    goal.end_y=6
send_goal_async(goal, feedback_callback) ---->  NavigateServer
spin_until_future_complete()                     execute_callback()
goal_handle.accepted = True                        validate start/end
        |                                          run BFS
        v                                          path found: 14 steps
get_result_async()                                 loop i=0..14:
spin_until_future_complete()                         fill feedback
        |                                            publish_feedback()
        |  <-- feedback arrives every 0.5s           time.sleep(0.5)
        |  feedback_callback() fires              goal_handle.succeed()
        |  prints: Step 1 | DOWN | (1,2)         return result
        |  prints: Step 2 | DOWN | (1,3)
        |  ...
        |  prints: Step 14 | DOWN | (8,6)
        |
        v
result_future complete
result.success     = True
result.total_steps = 14
result.path_x      = [1,1,1,...]
result.path_y      = [1,2,3,...]
        |
        v
print NAVIGATION COMPLETE
draw_grid()
destroy_node()
rclpy.shutdown()
```

14 Topics vs Services vs Actions — Full Summary

	Topic	Service	Action
Pattern	Publish / Subscribe	Request / Response	Goal / Feedback / Result
Parts	1 (message)	2 (request, response)	3 (goal, feedback, result)
File type	`.msg`	`.srv`	`.action`
Duration	Continuous	Instant	Long running
Feedback	Continuous stream	None	Periodic while running
Cancellable	N/A	No	Yes
Robot use	Sensor data, velocity	Query, compute, toggle	Navigate, grasp, scan
This series	Not covered here	Return full path list	Execute path step by step

15 Common Mistakes & Fixes

Mistake	Error	Fix
Comments in `.action` file	`NameError: invalid field name`	Remove all `#` lines
Missing `direction` or `status` in `.action`	`AttributeError: no attribute 'direction'`	Ensure all 6 feedback fields are in the file
Forgot to rebuild interfaces after editing `.action`	Old class used, fields missing	`colcon build pathfinder_interfaces` then source
Forgot `goal_handle.succeed()` or `abort()`	Client waits forever	Always call one of: succeed, abort, canceled
Forgot `return result` in server	Client hangs	Always `return result` at every exit point
Wrong argument count to client	`IndexError`	Pass all 4 args: `x1 y1 x2 y2`
`OBSTACLES` mismatch between server and client	Grid drawn incorrectly	Keep both sets identical
Build order wrong	Import error on nodes	Always build interfaces before nodes

16 Quick Reference

16.1 Commands Cheat Sheet

```
# Build
colcon build --packages-select pathfinder_interfaces
colcon build --packages-select pathfinder_nodes
source install/setup.bash

# Run server
ros2 run pathfinder_nodes navigate_server

# Run client
ros2 run pathfinder_nodes navigate_client <x1> <y1> <x2> <y2>

# Action CLI tools
ros2 action list
ros2 action info /navigate_path
ros2 action send_goal --feedback /navigate_path \
  pathfinder_interfaces/action/NavigatePath \
  "{start_x: 1, start_y: 1, end_x: 8, end_y: 6}"
```

16.2 Test Cases Summary

#	Start	End	Expected	Tests
1	(1,1)	(8,6)	SUCCESS	Standard navigation with feedback
2	(0,0)	(9,9)	SUCCESS	Long path, corner to corner
3	(0,0)	(5,0)	5 steps RIGHT	Straight horizontal path
4	(0,3)	(9,3)	SUCCESS	Navigate around row obstacles
5	(2,1)	(8,6)	ABORTED	Start on obstacle
6	(0,0)	(4,5)	ABORTED	End on obstacle
7	(3,3)	(3,3)	0 steps	Already at destination
8	(0,0)	(15,15)	ABORTED	Out of bounds
9	(1,1)	(8,6)	SUCCESS	CLI with live feedback
10	(0,0)	(0,9)	9 steps DOWN	Straight vertical path

End of Tutorial — ROS 2 Actions: Pathfinder with Live Feedback